

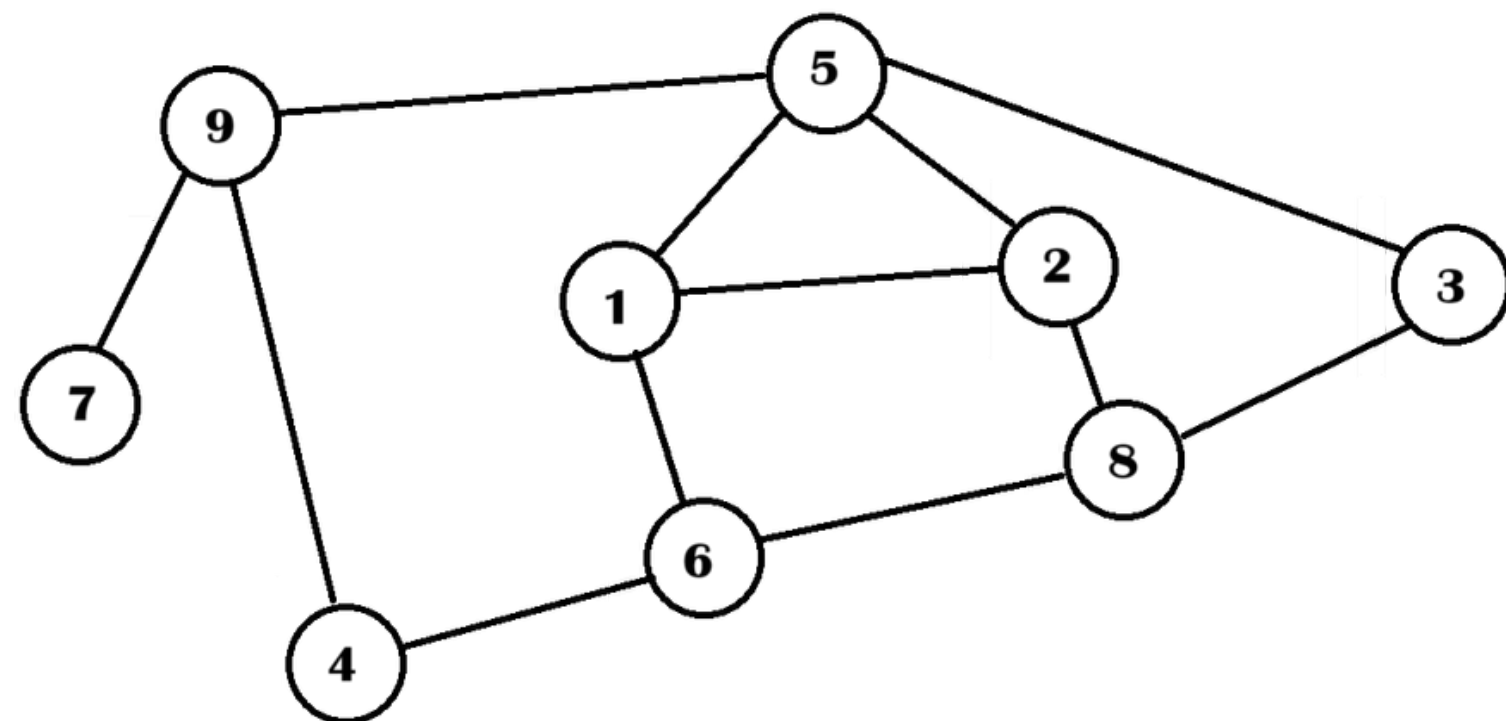
CLASE 5

BÚSQUEDA EN PROFUNDIDAD (DFS)

RECUERDO: LISTA DE ADYACENCIA

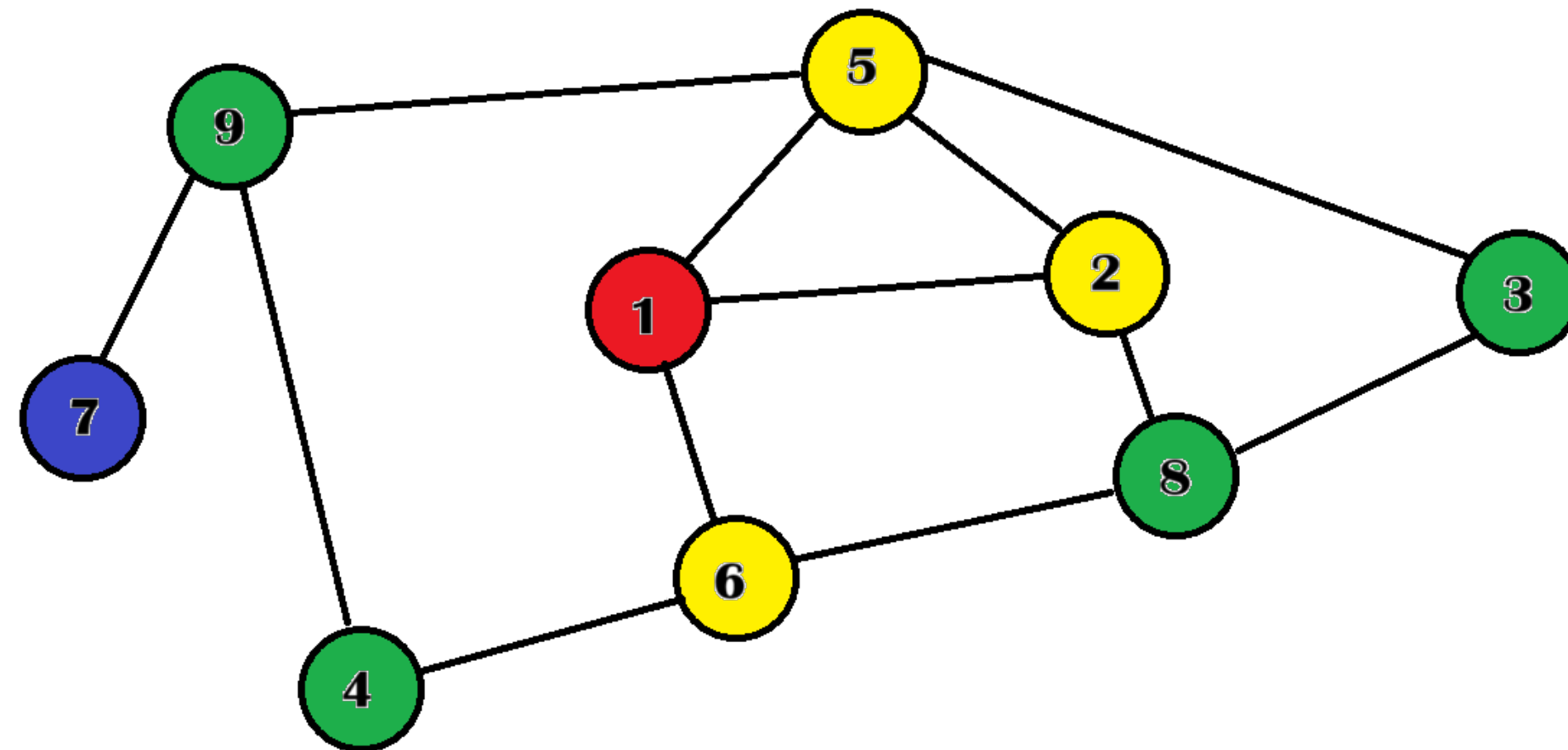
Guardamos para cada nodo una lista con los nodos que son adyacentes a él. Esta es muy conveniente ya que nos permite iterar sobre las aristas y acceder a las aristas de un nodo en particular.

1: [2, 5, 6]
2: [1, 5, 8]
3: [5, 8]
4: [6, 9]
5: [1, 2, 3, 9]
6: [1, 4, 8]
7: [9]
8: [2, 3, 6]
9: [4, 5, 7]



RECUERDO: BFS

En la búsqueda en anchura comenzamos en un nodo y visitamos todos sus vecinos, y luego los vecinos de los vecinos, y así sucesivamente....

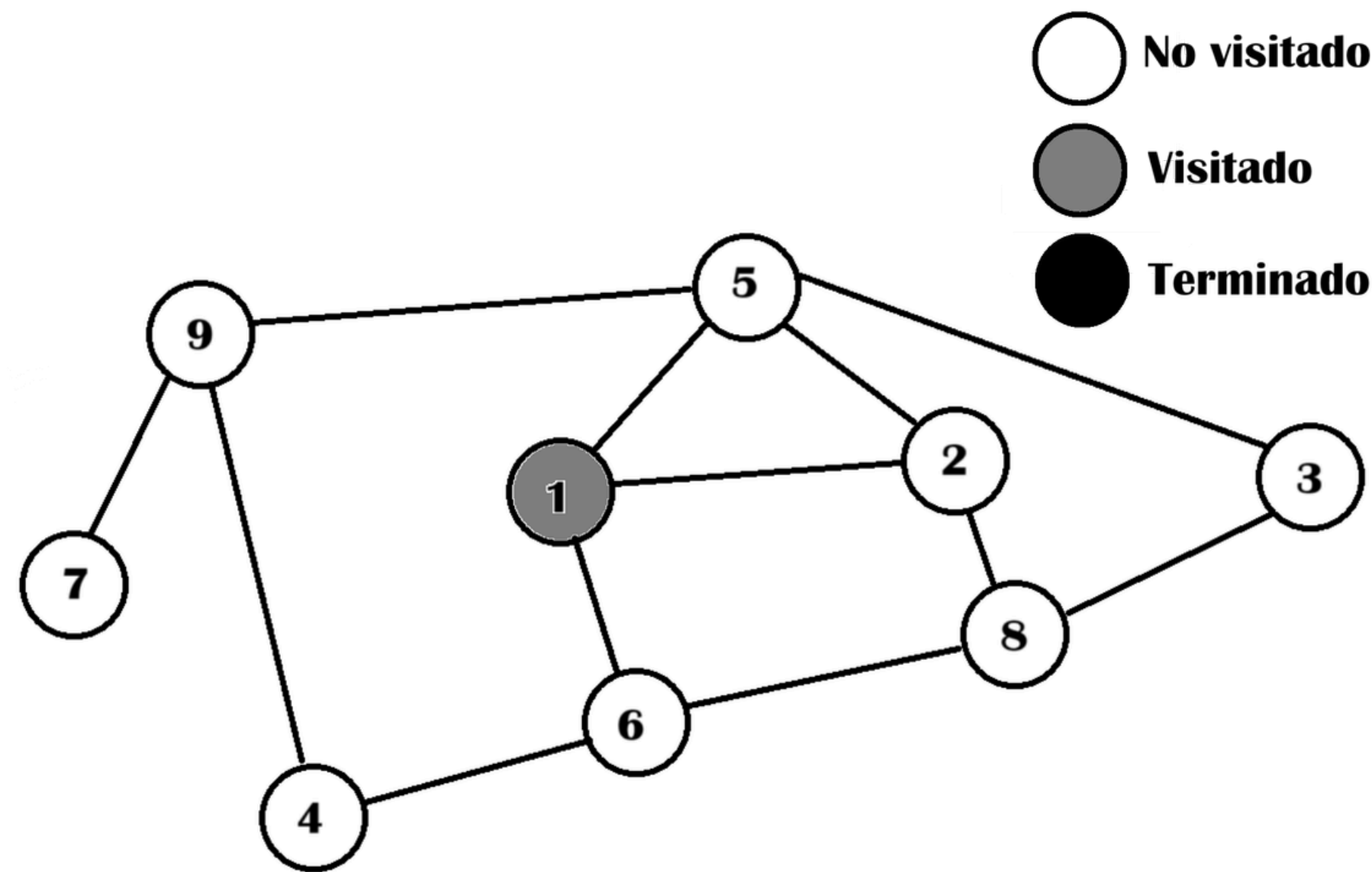


NUEVA ESTRATEGIA

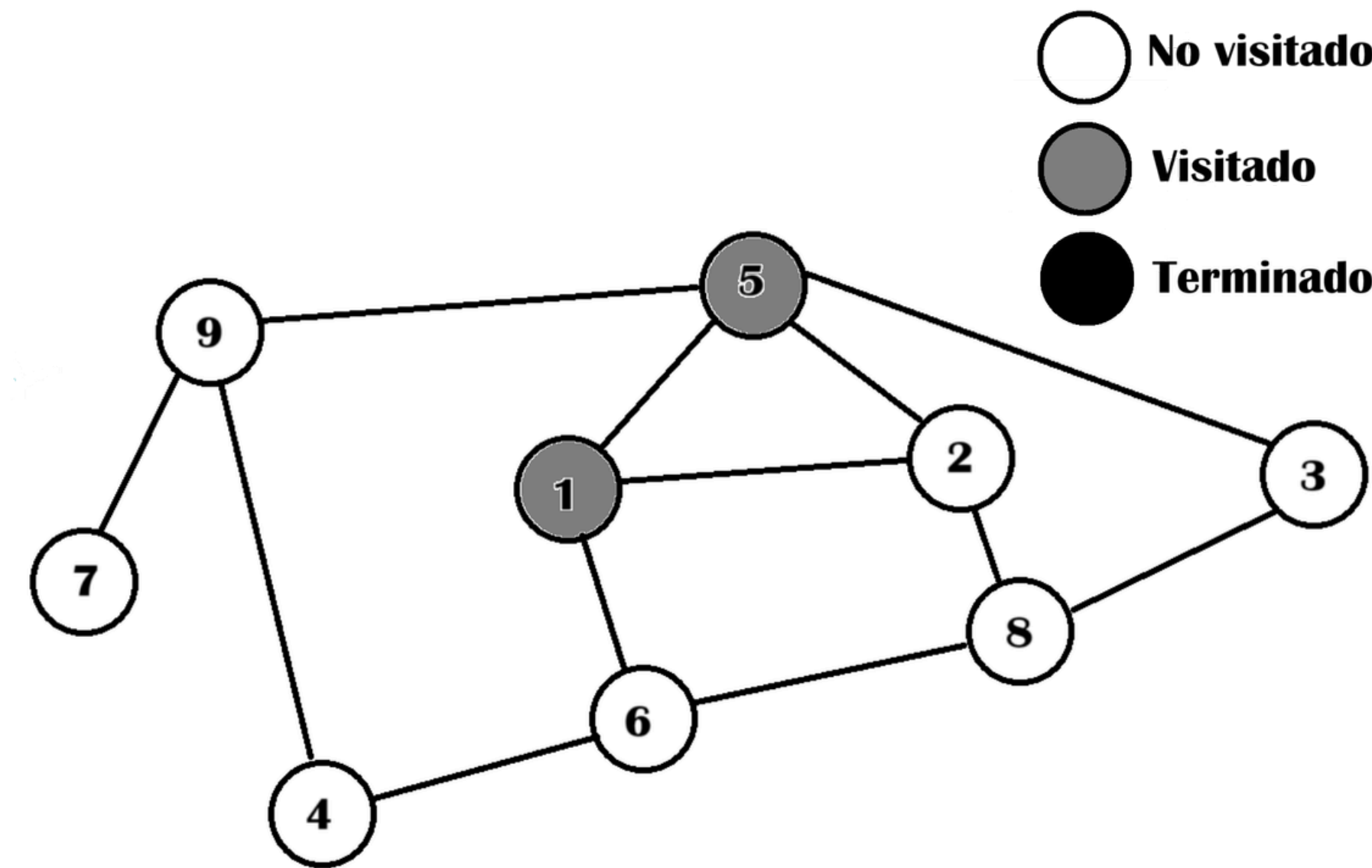
En vez de procesar un nodo completamente, cada vez que encuentro un nuevo nodo lo dejo a la mitad y sigo con el nuevo.

Veamos con un ejemplo

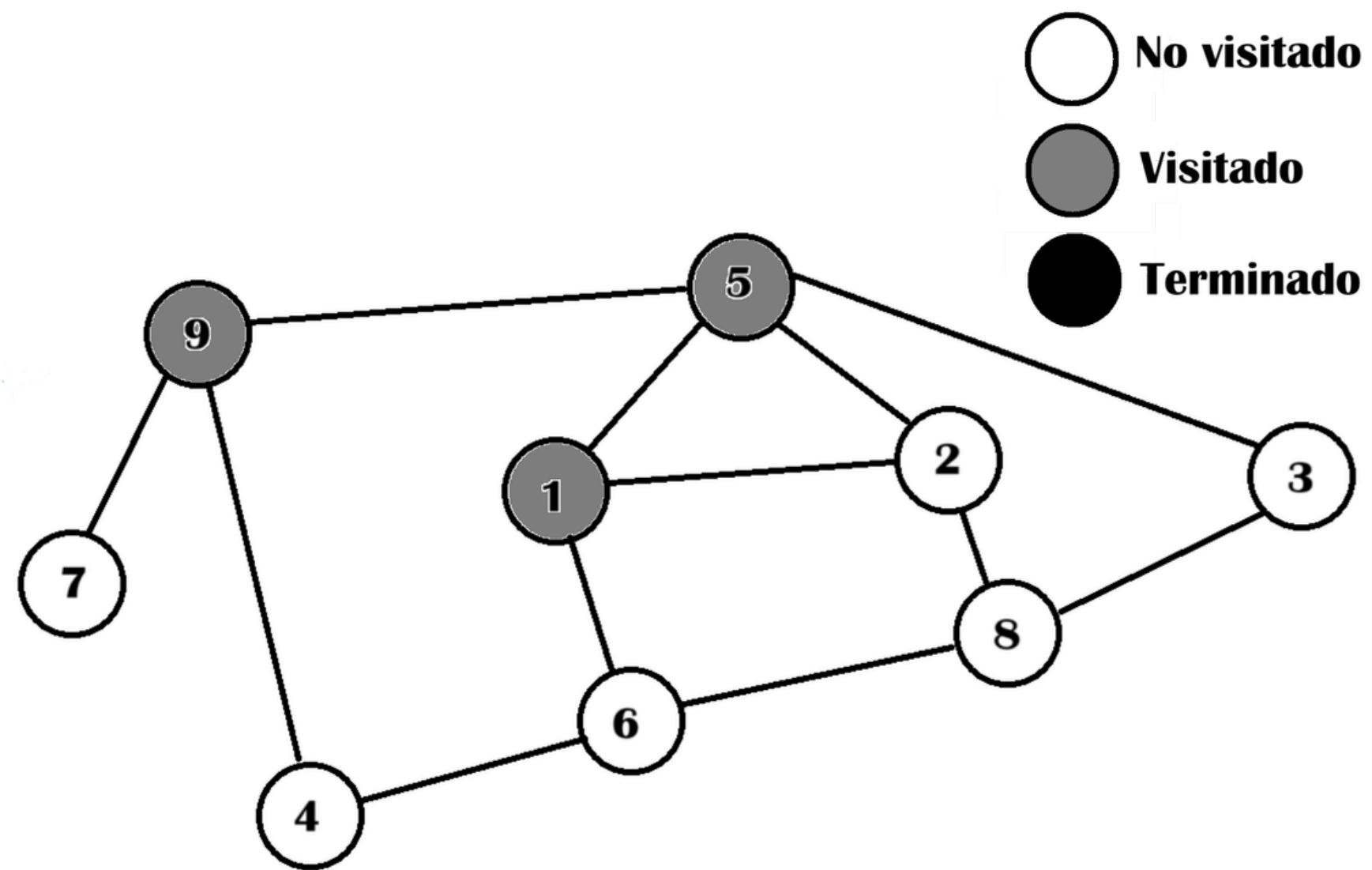
EJEMPLO



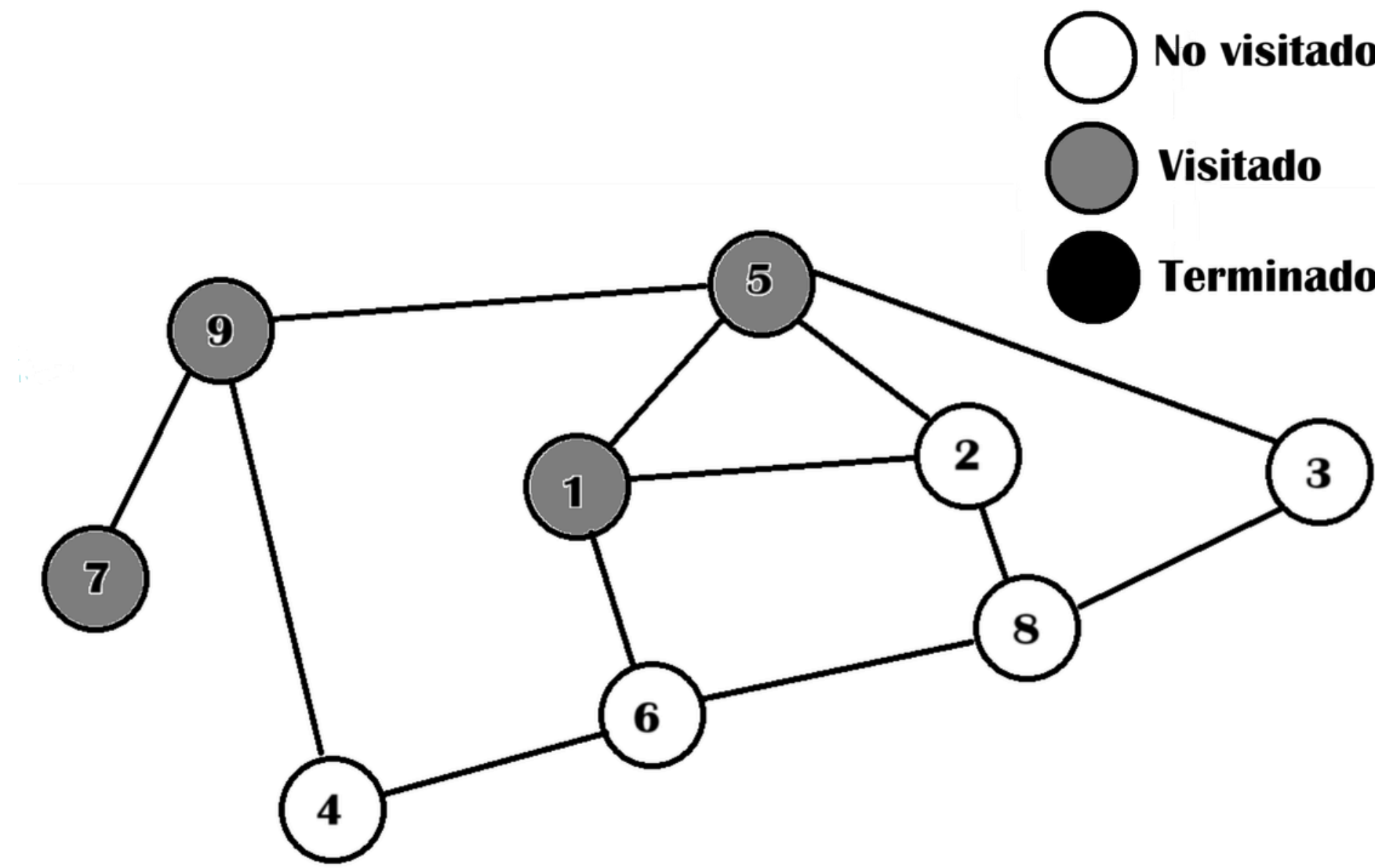
EJEMPLO



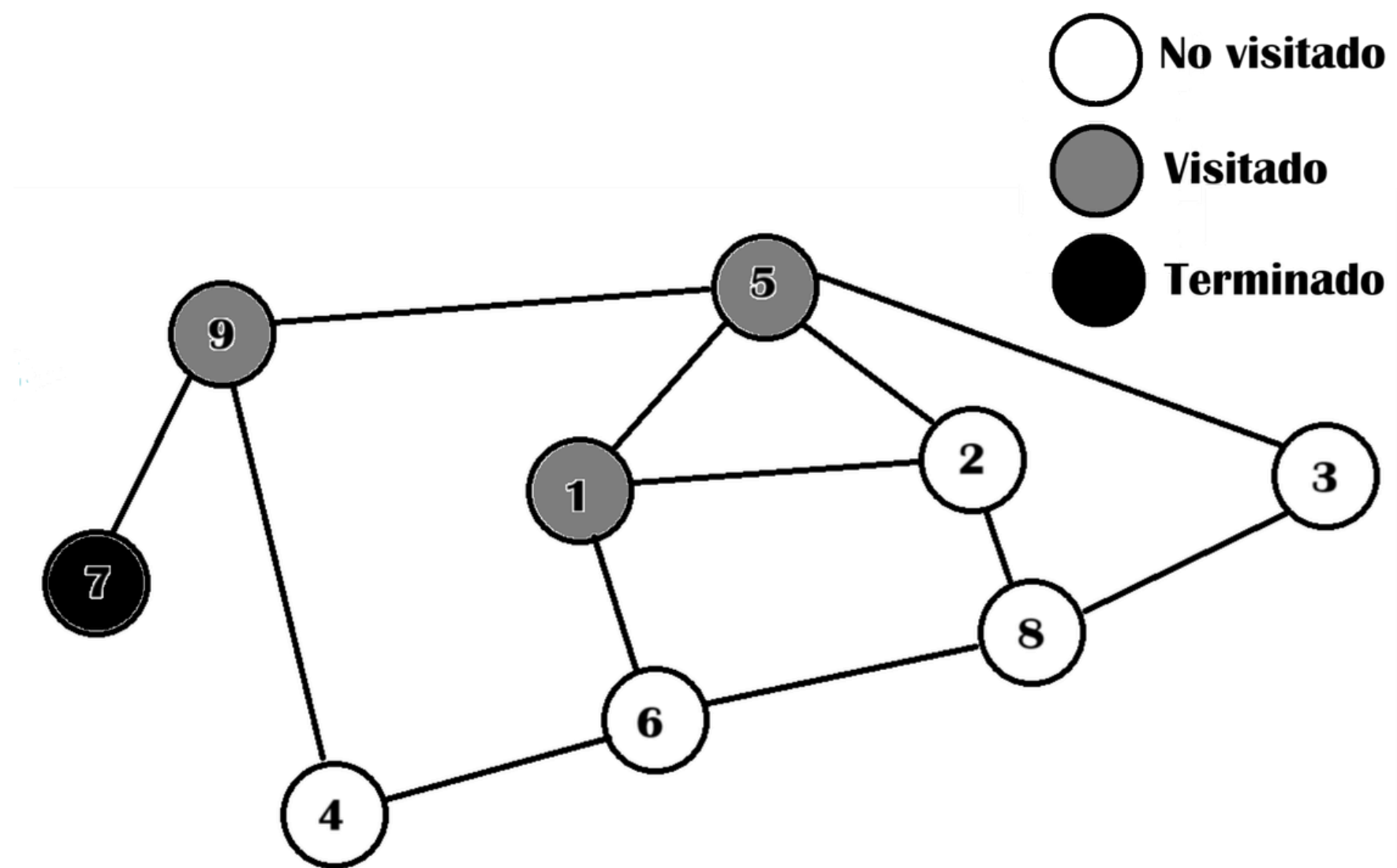
EJEMPLO



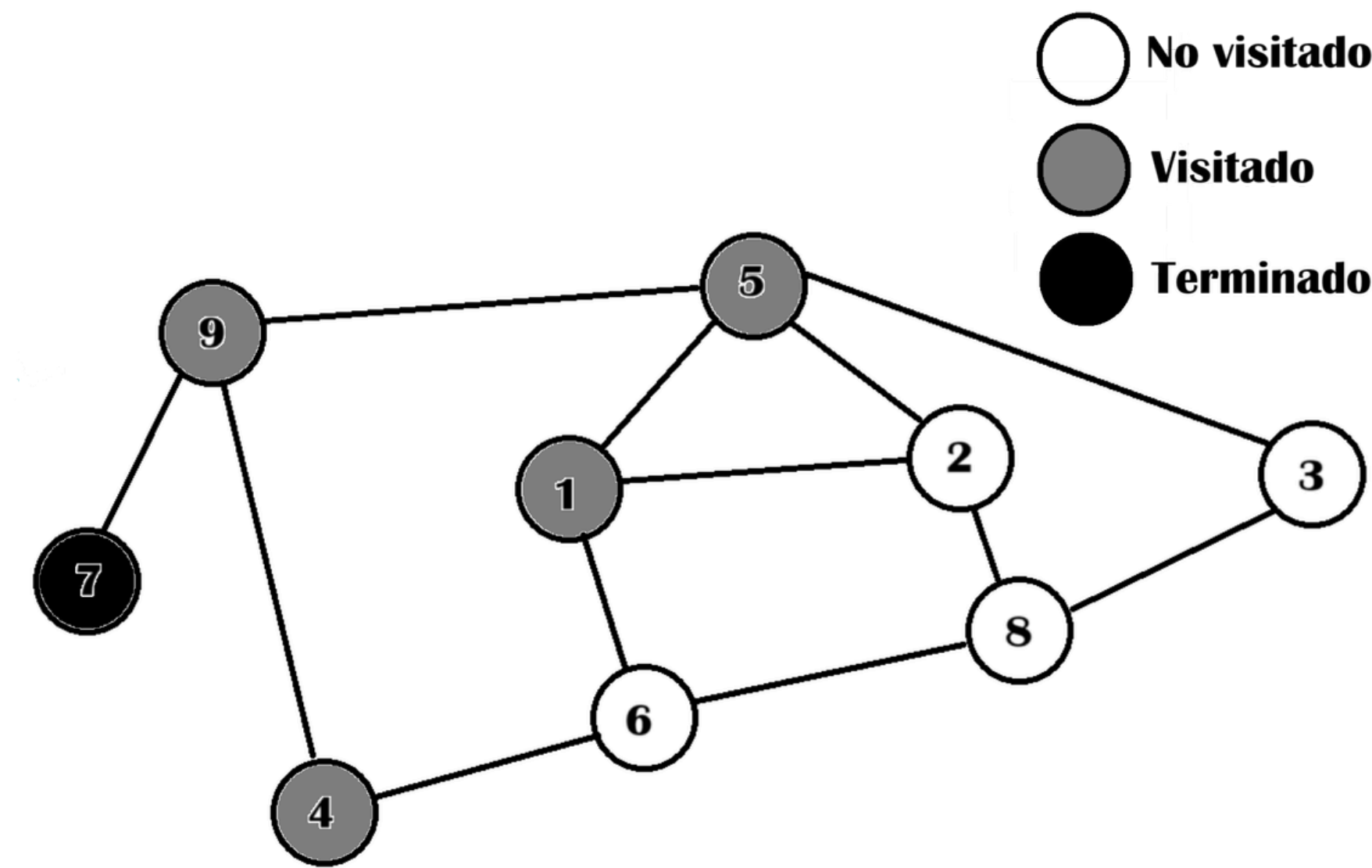
EJEMPLO



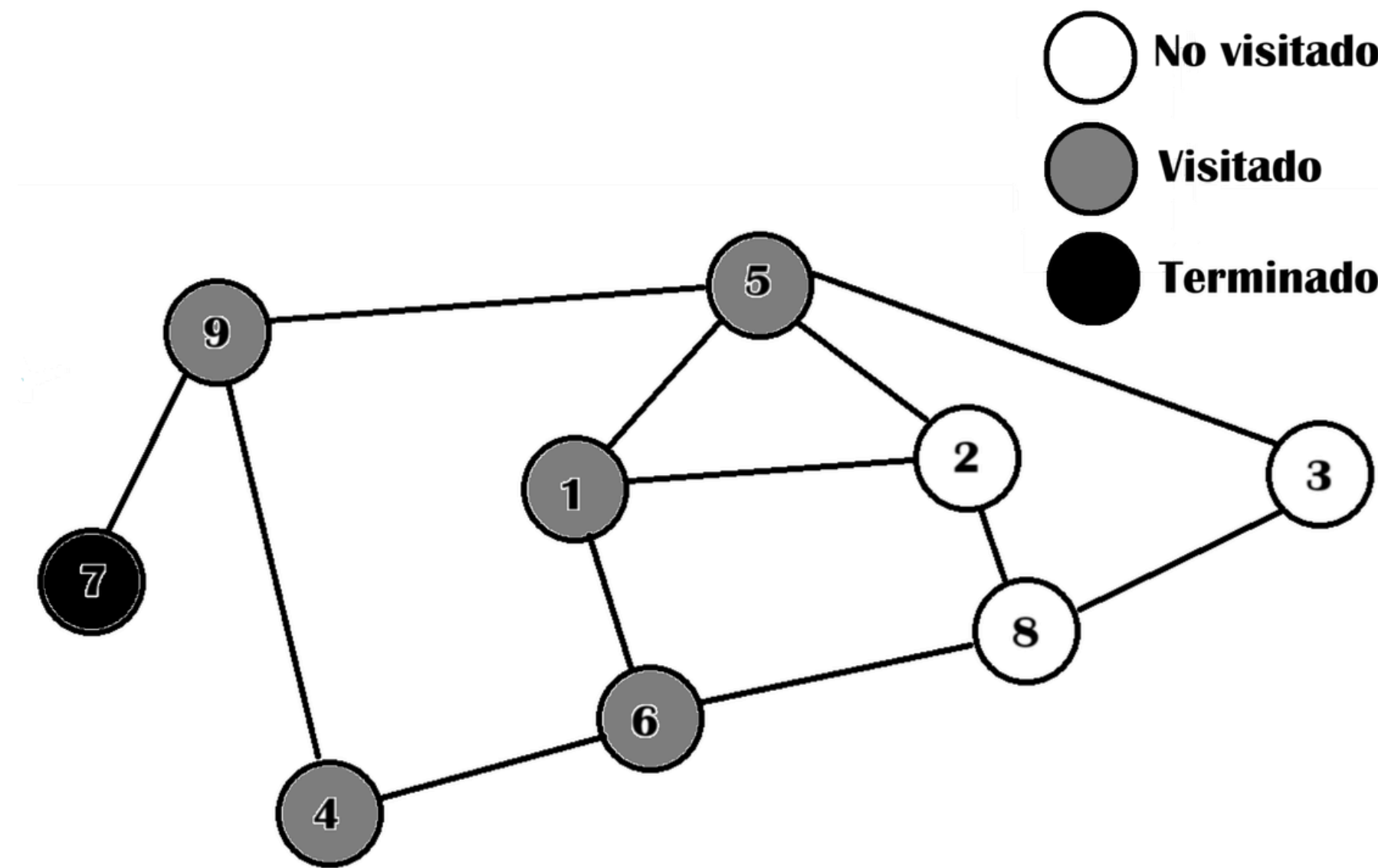
EJEMPLO



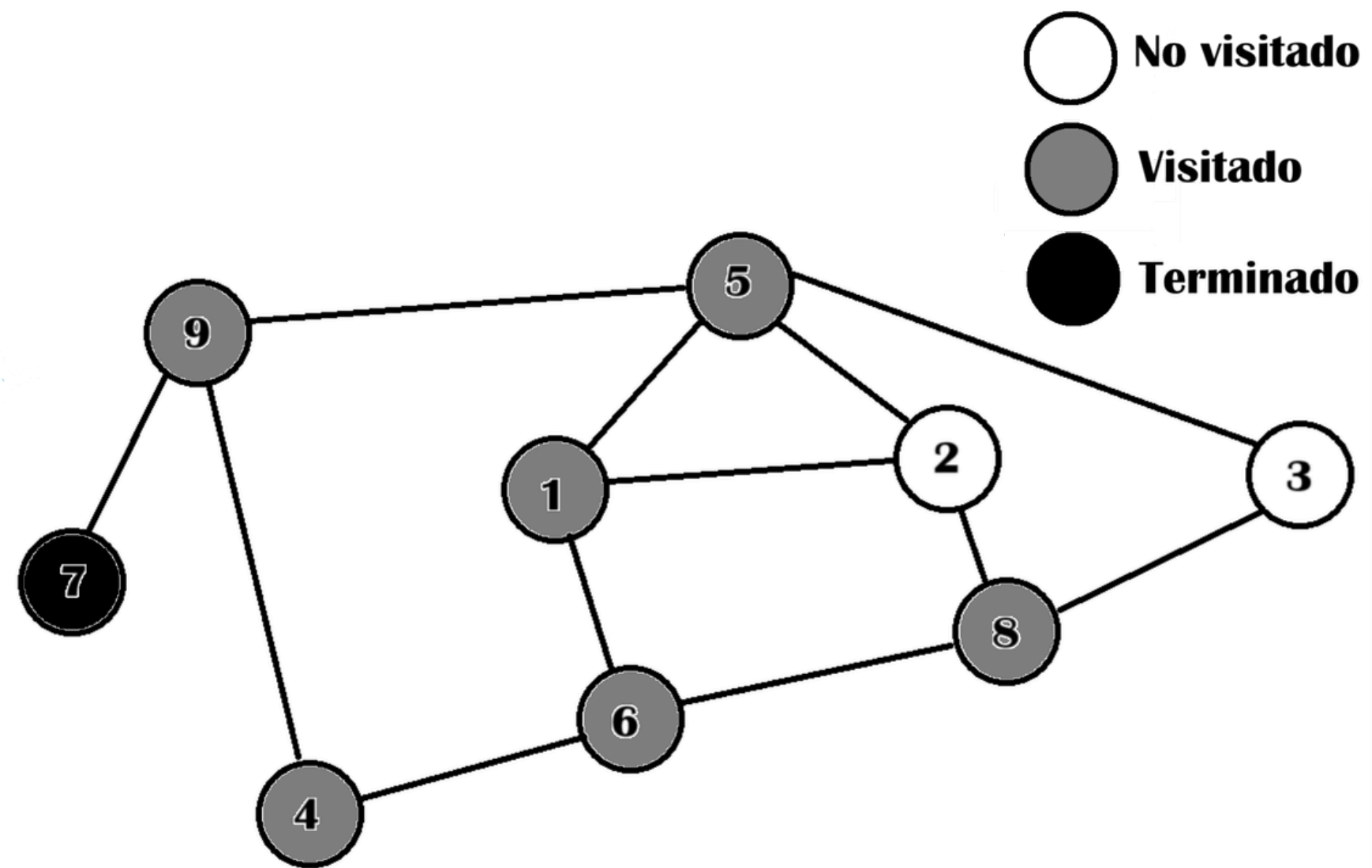
EJEMPLO



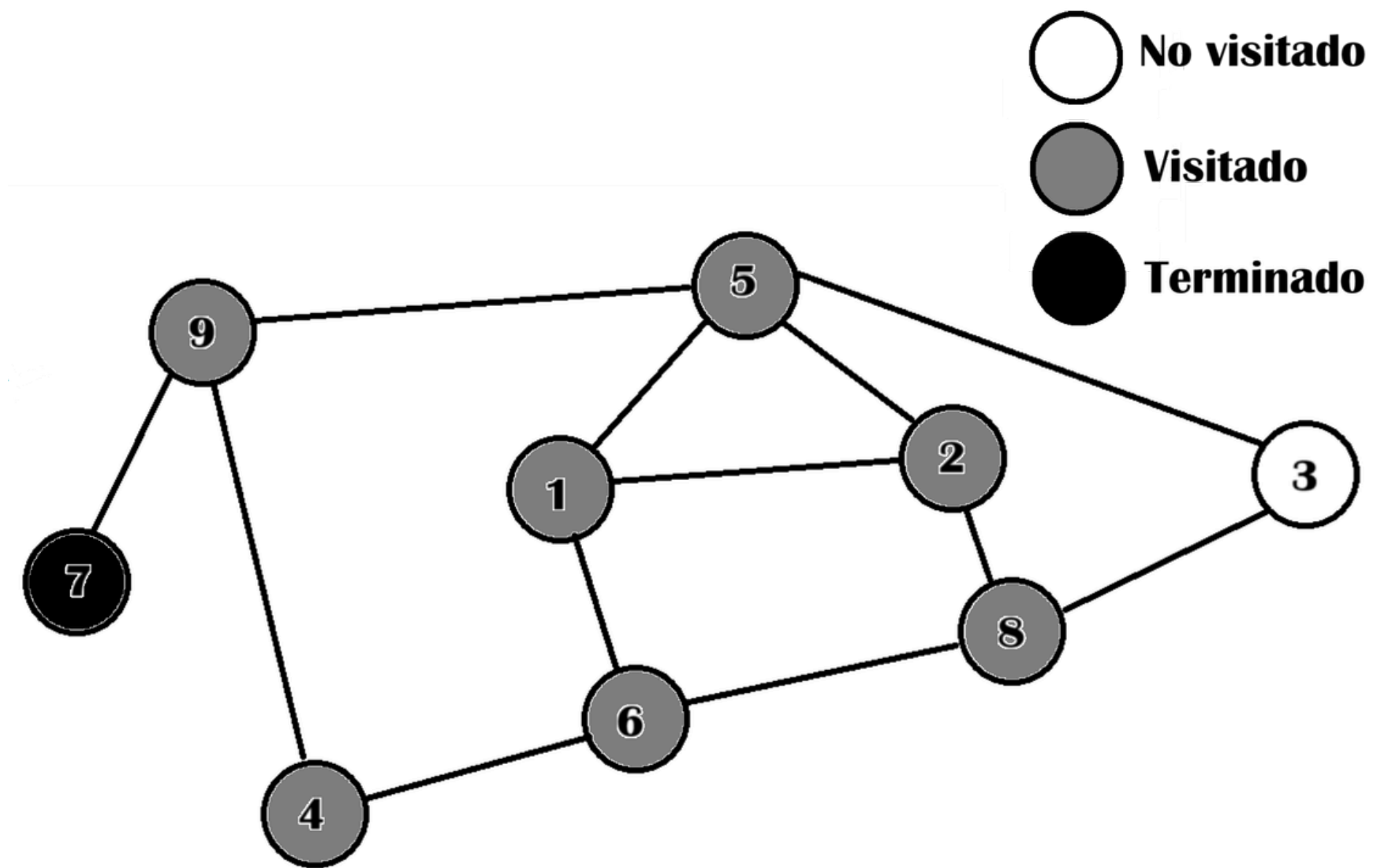
EJEMPLO



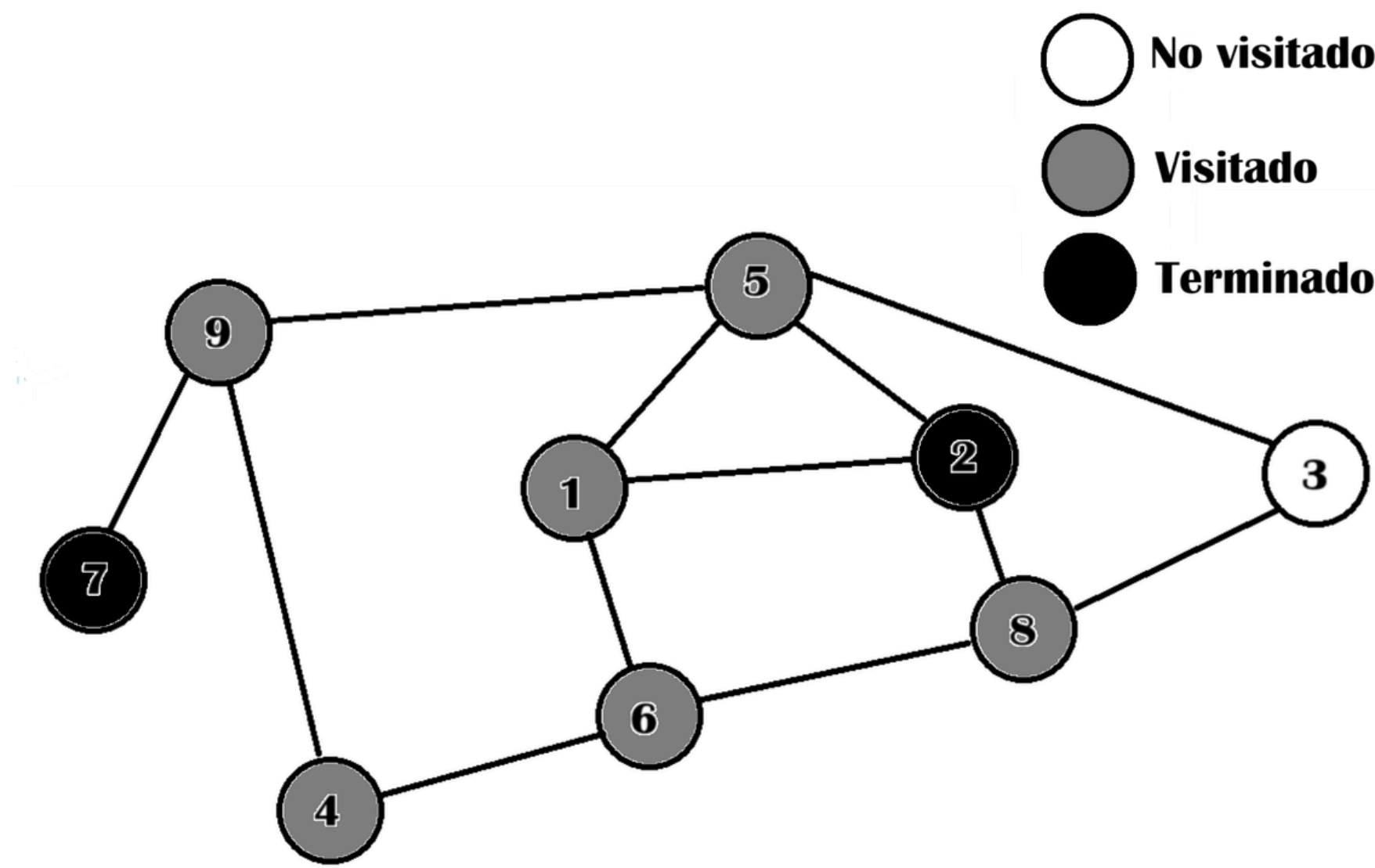
EJEMPLO



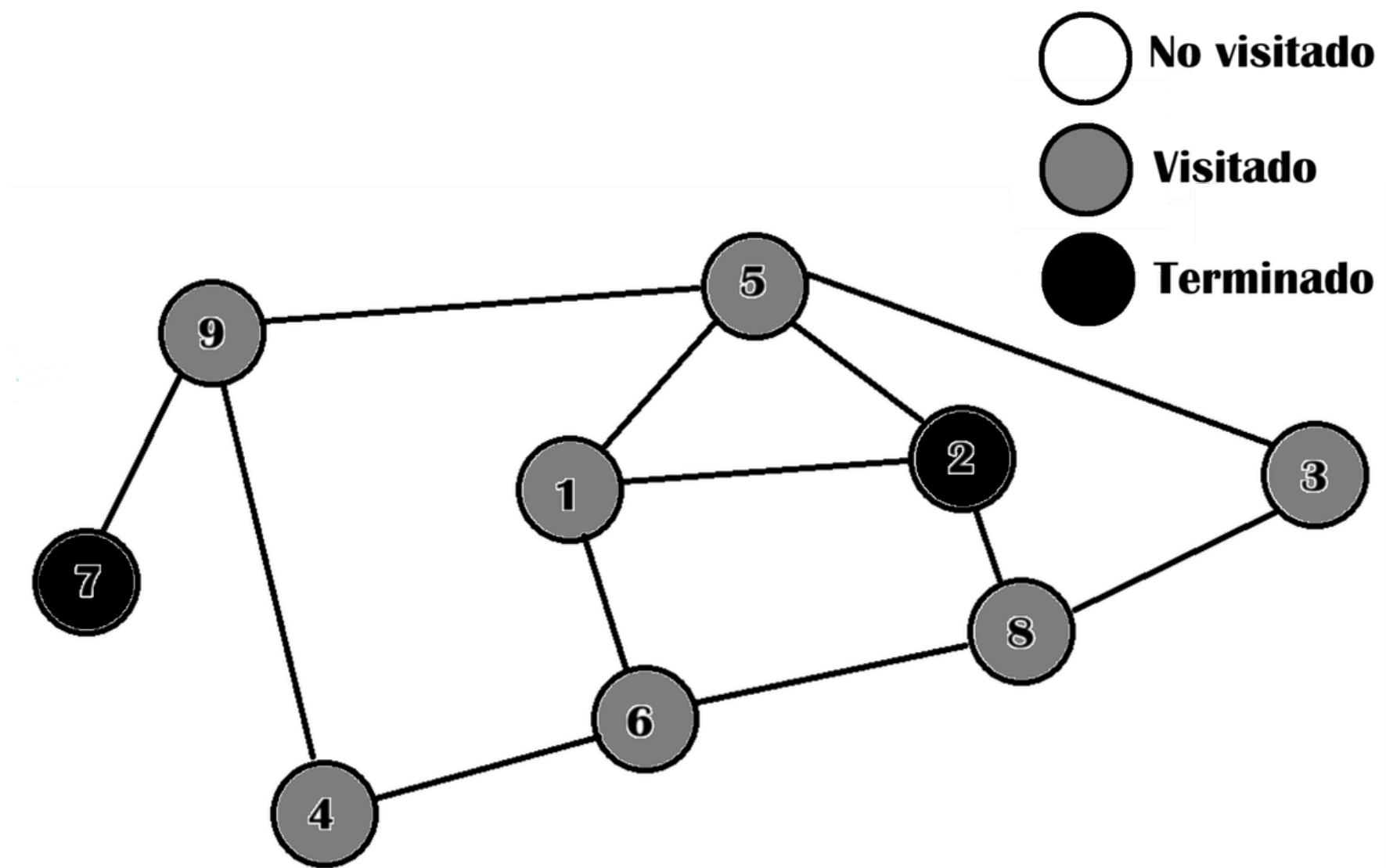
EJEMPLO



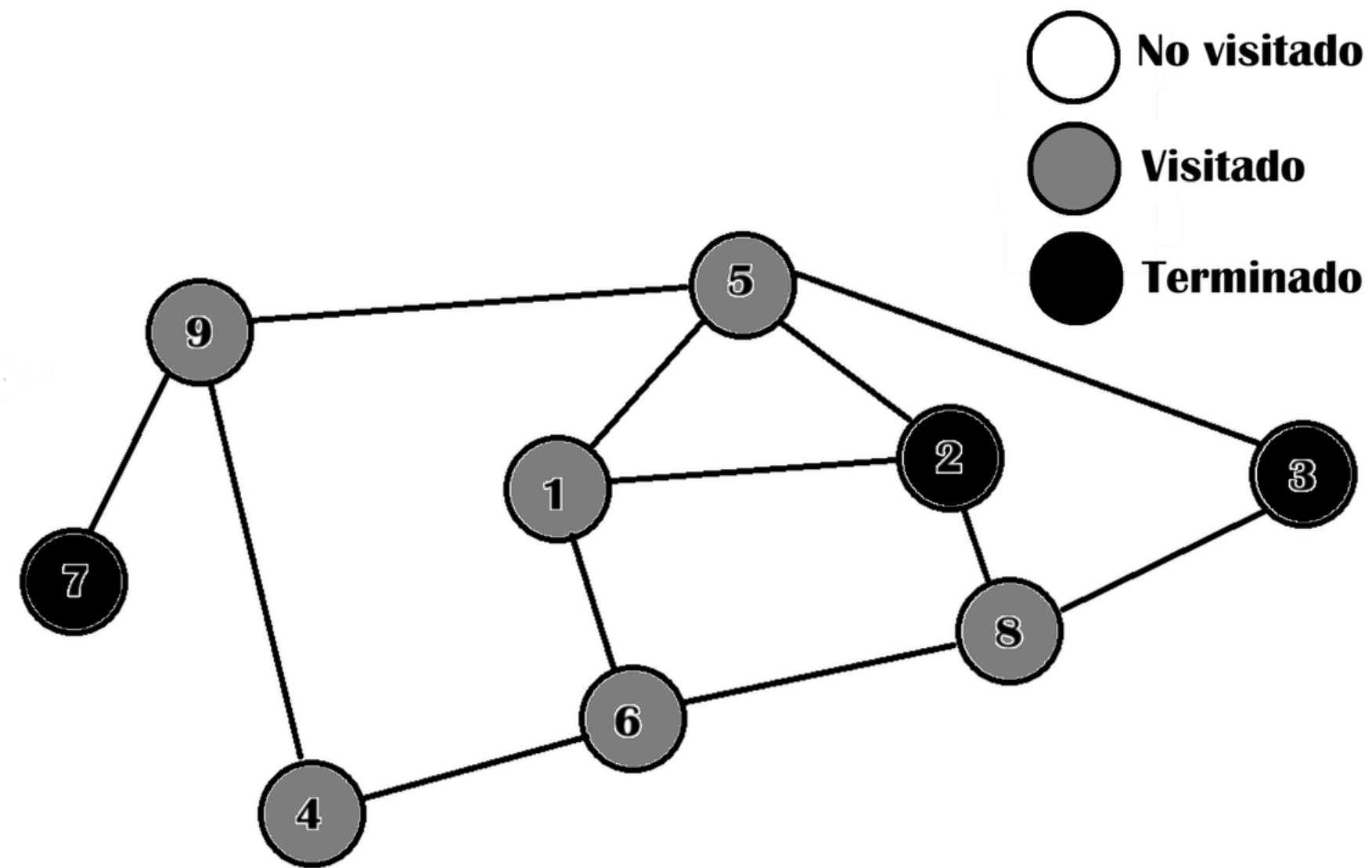
EJEMPLO



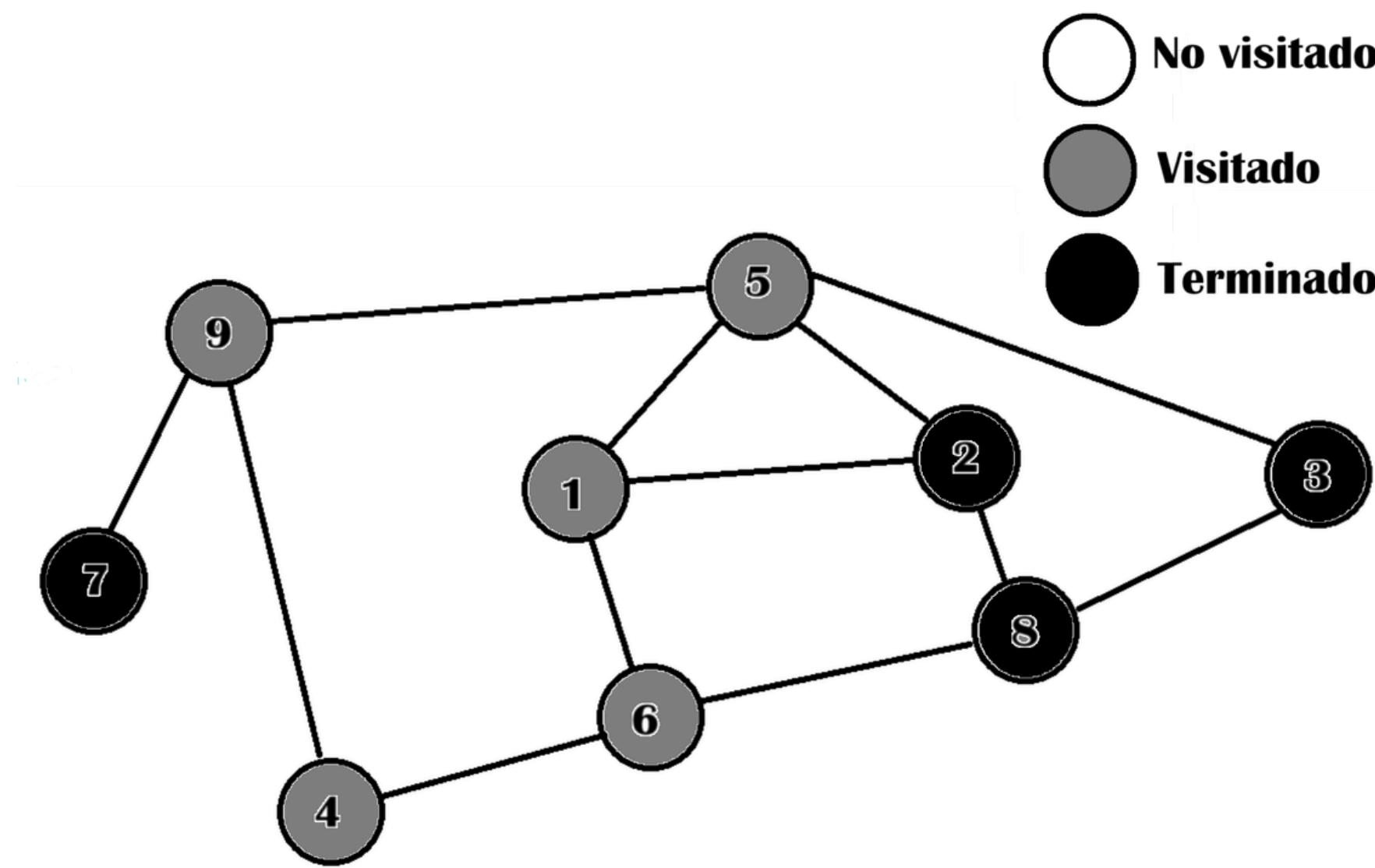
EJEMPLO



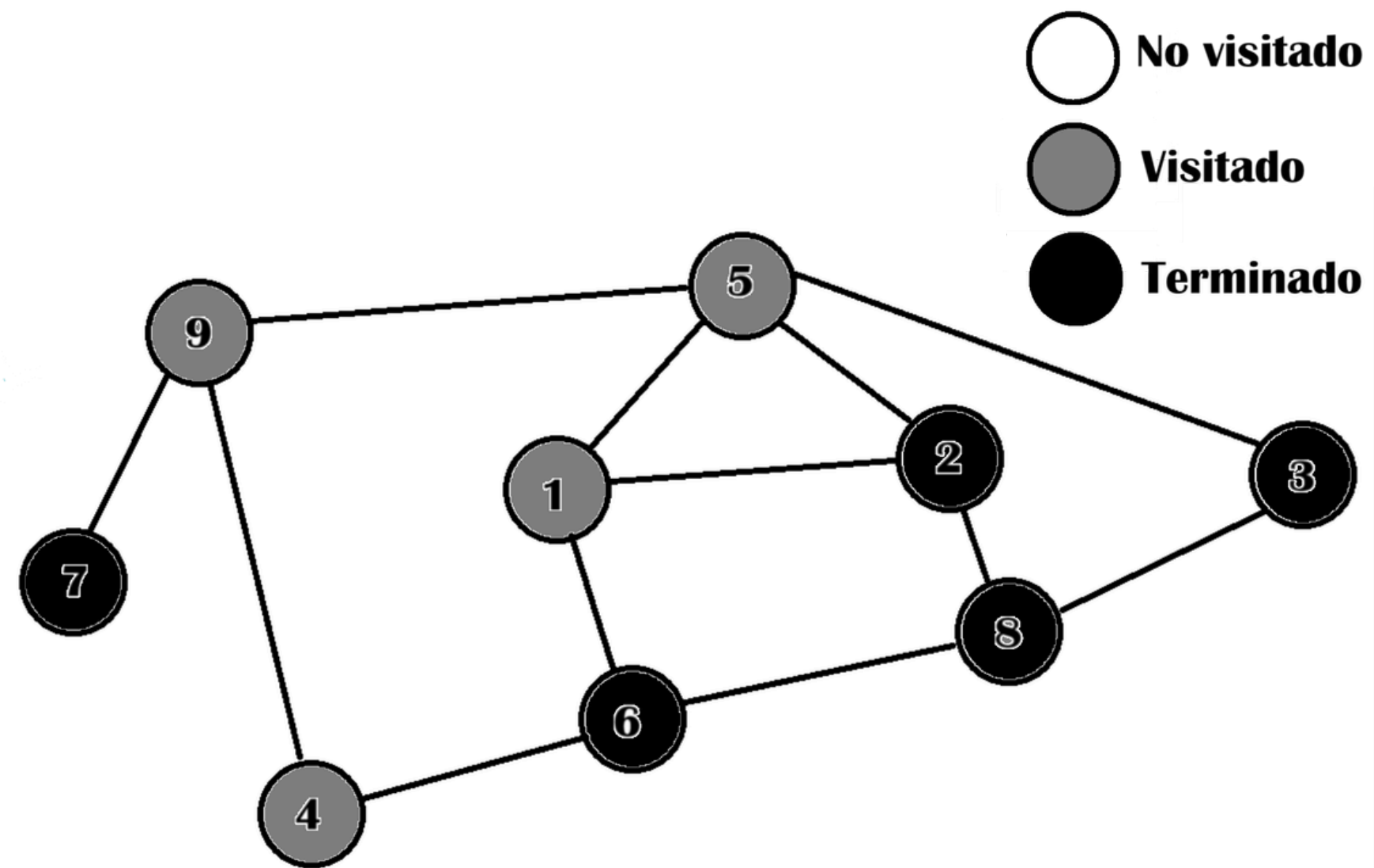
EJEMPLO



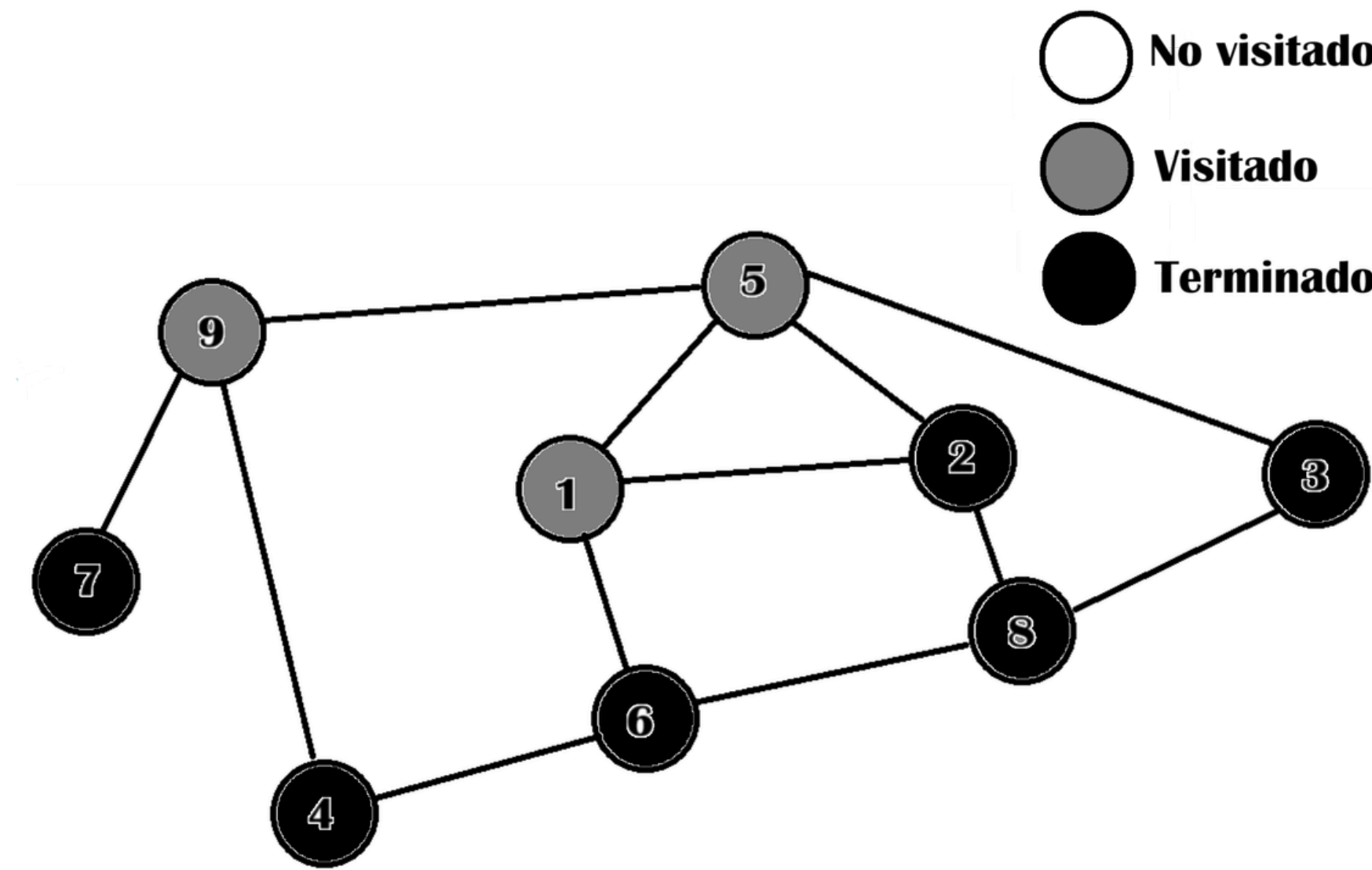
EJEMPLO



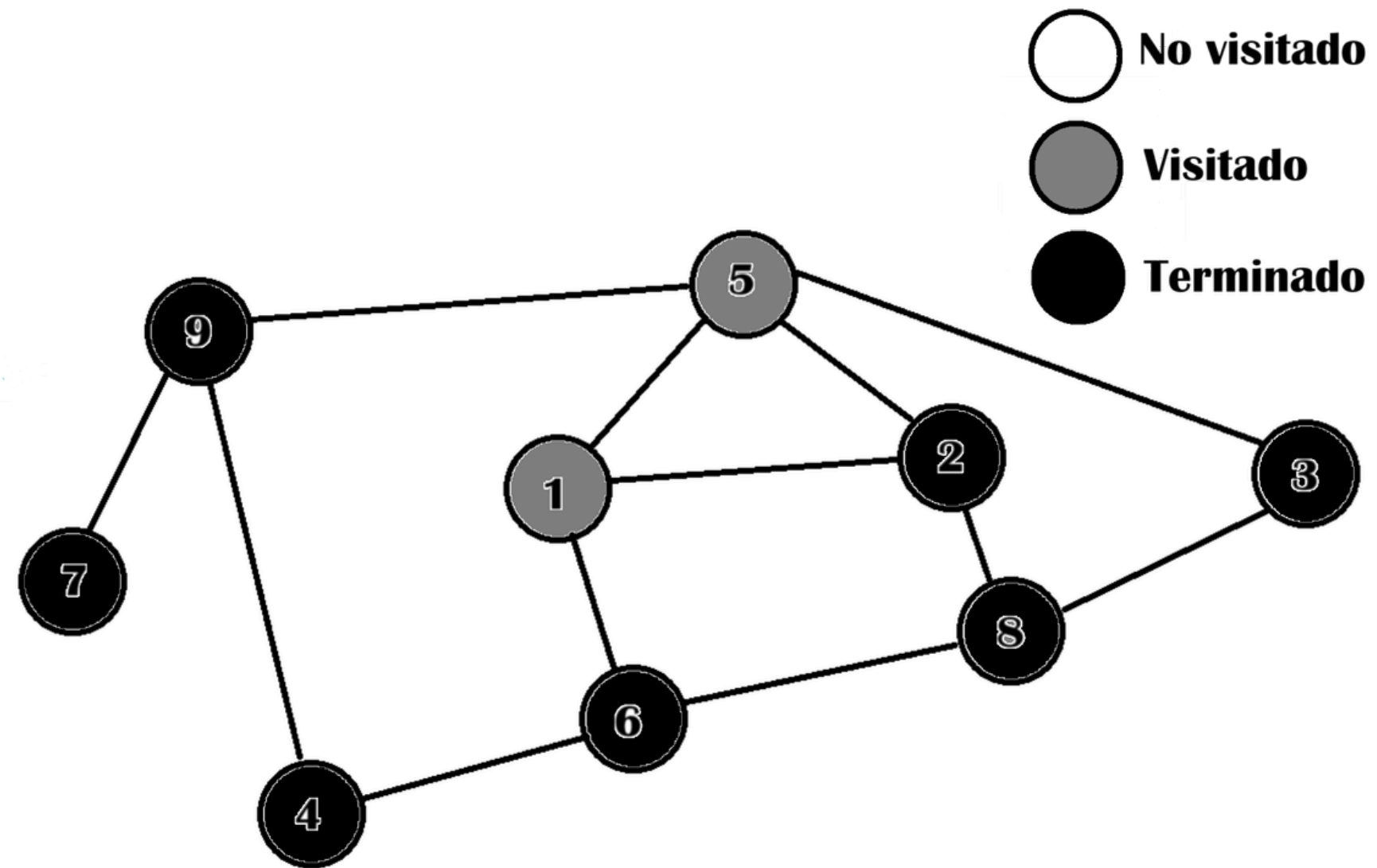
EJEMPLO



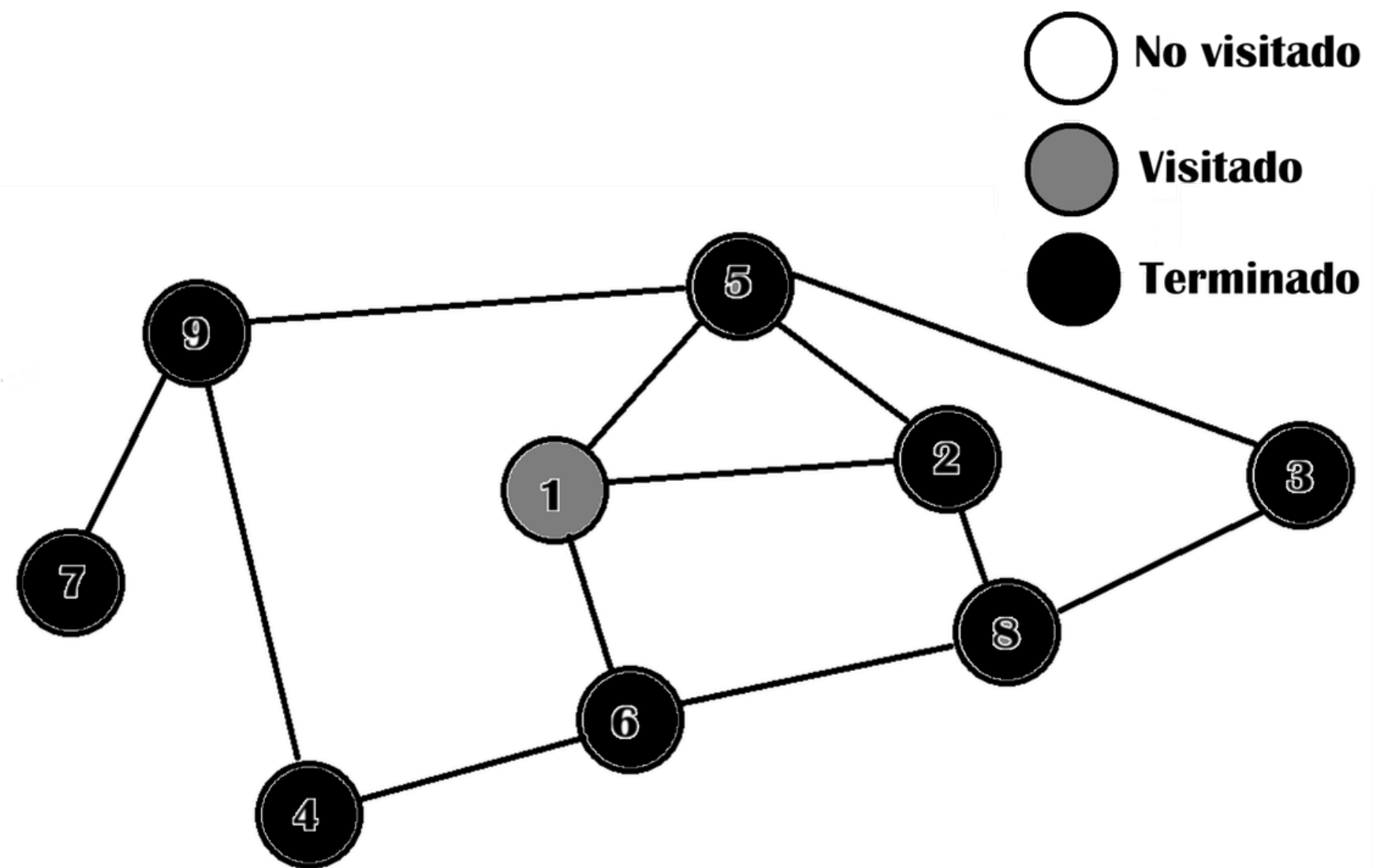
EJEMPLO



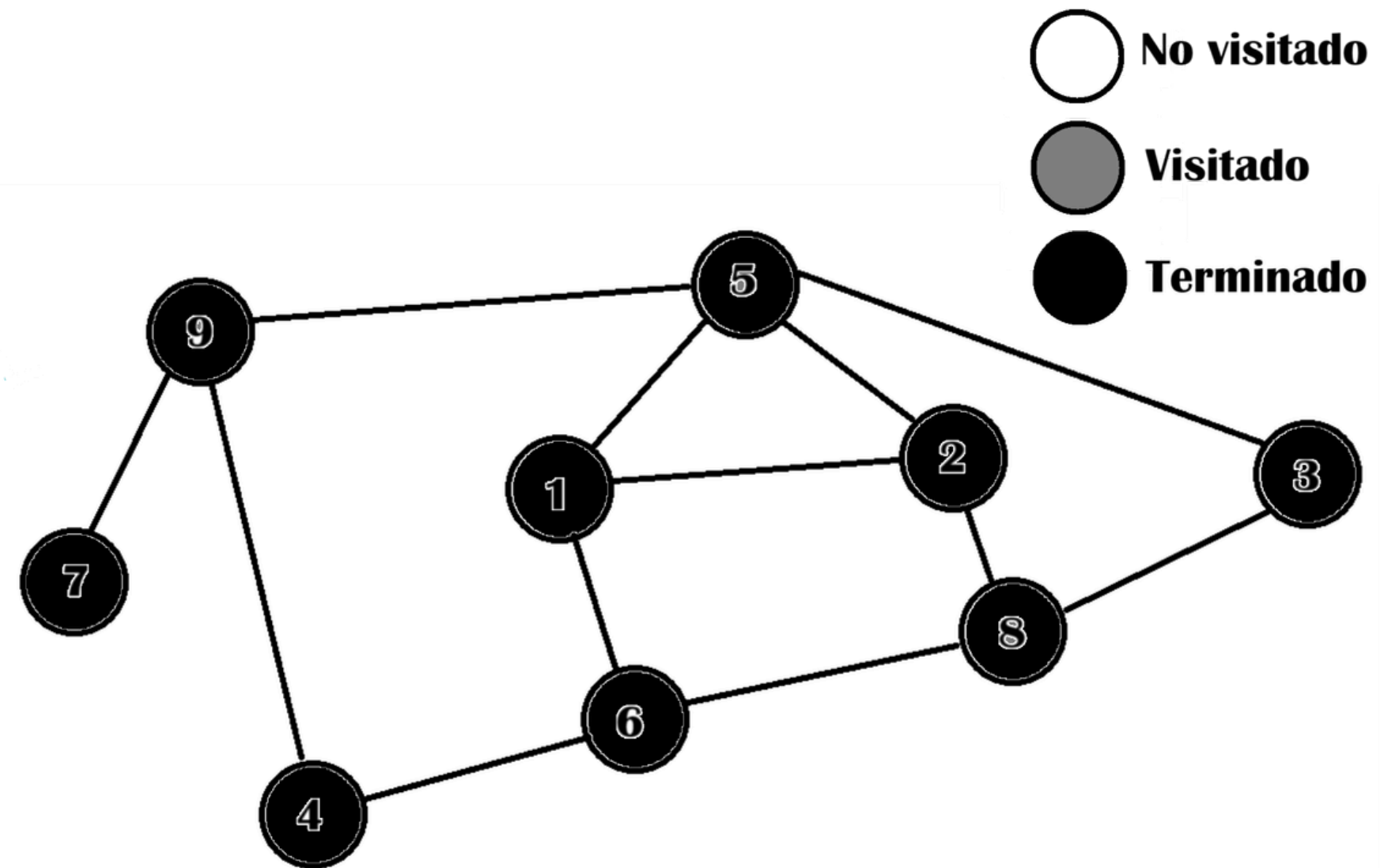
EJEMPLO



EJEMPLO



EJEMPLO



IMPLEMENTACION

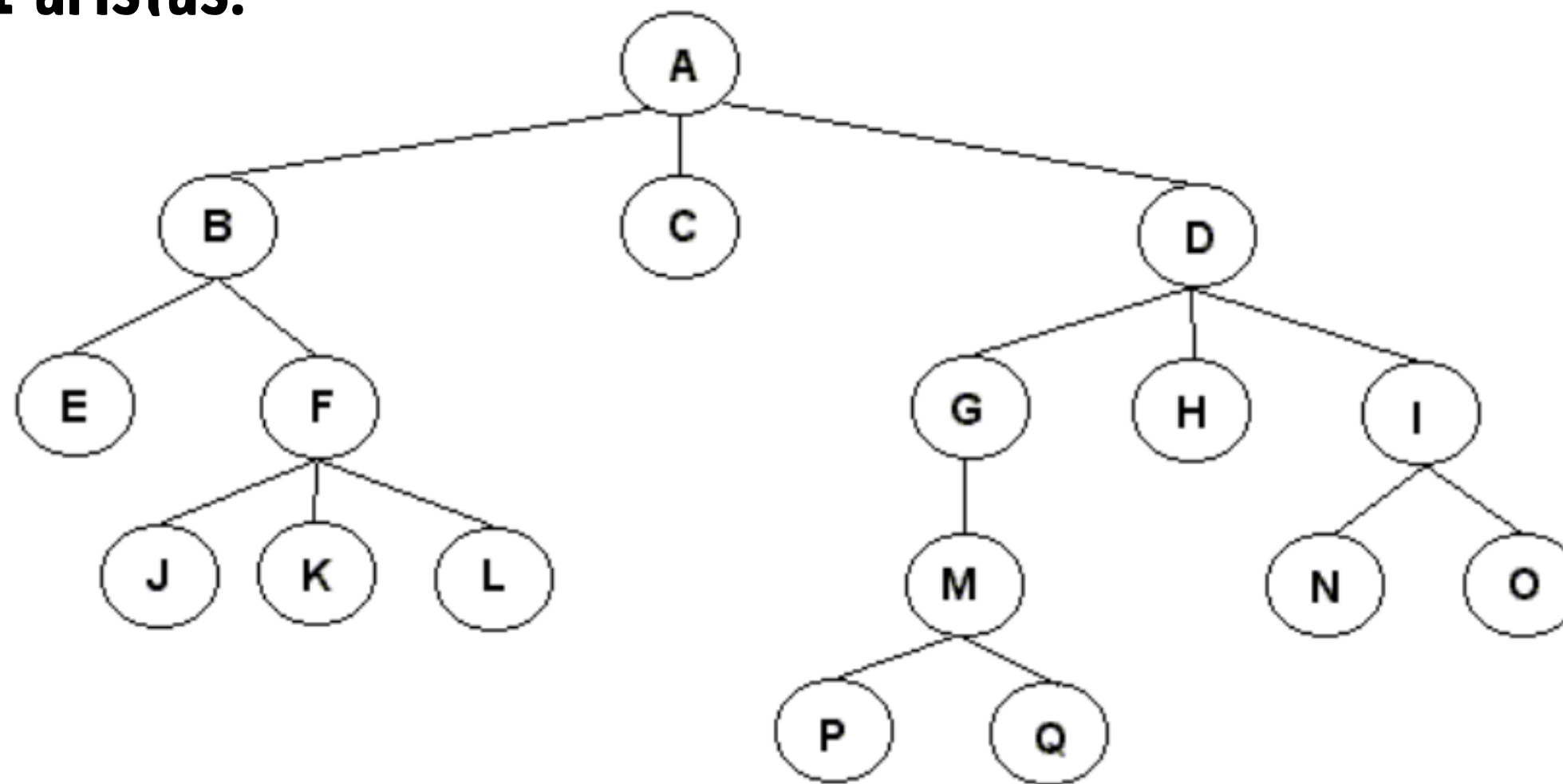
La implementación más sencilla es con una función recursiva

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  vector<vector<int> > edges;
4  vector<int> visitado;
5
6  void dfs(int nodo){
7      visitado[nodo] = 1;
8      for(auto u: edges[nodo]){
9          if(visitado[u] == 0){
10             dfs(u);
11         }
12     }
13     visitado[nodo] = 2;
14 }
15
```

```
16  int main(){
17
18      int n, m;
19      cin>>n>>m;
20
21      edges.resize(n);
22      visitado.resize(n, 0);
23
24      for(int i = 0; i<m; i++){
25          int u, v;
26          cin>>u>>v;
27          u--; v--;
28          edges[u].push_back(v);
29          edges[v].push_back(u);
30      }
31
32      dfs(0);
33
34 }
```

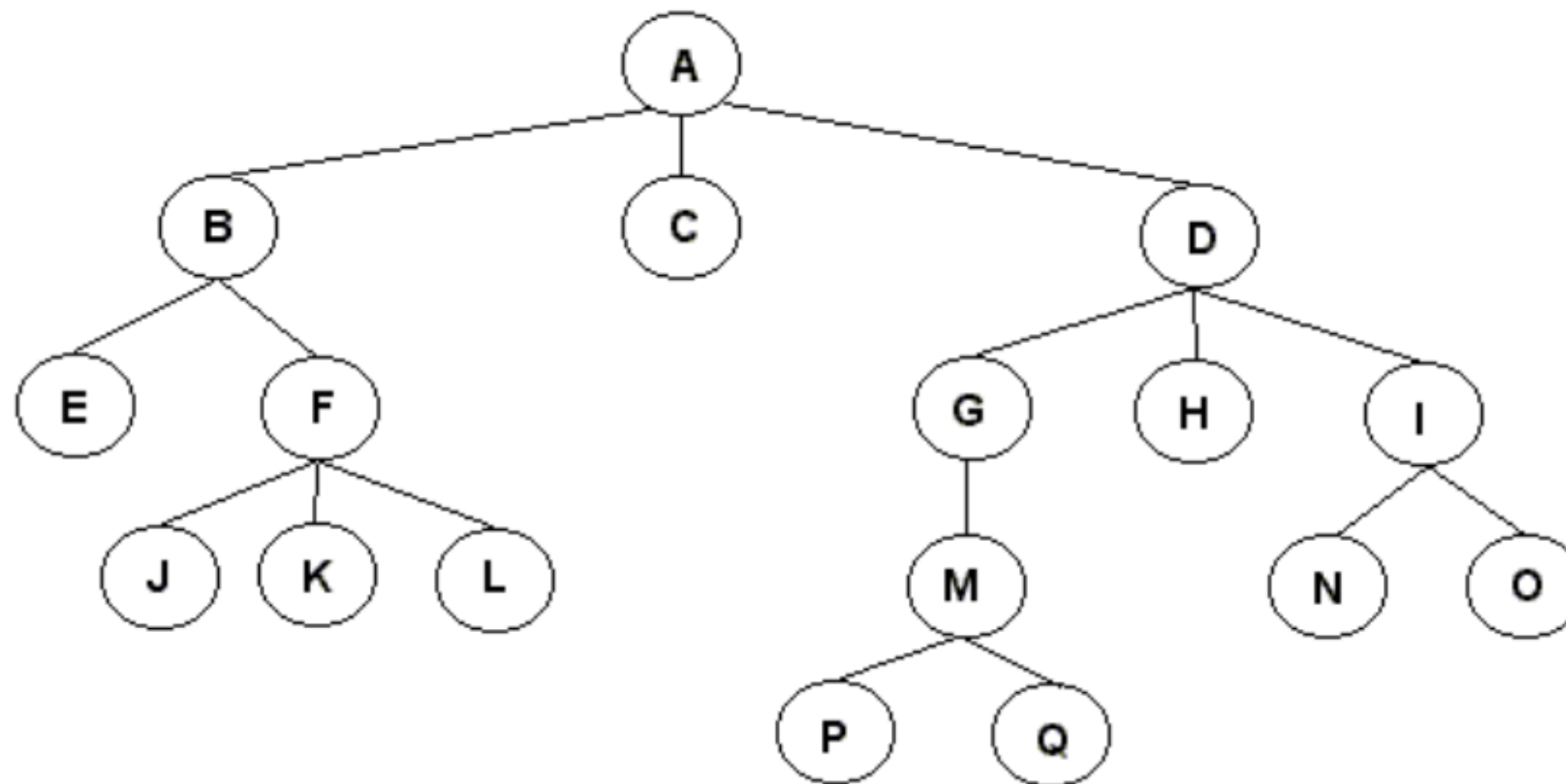
ÁRBOLES

Un árbol es un grafo conexo y sin ciclos. Se puede demostrar que un árbol tiene $n-1$ aristas.



DFS SOBRE ÁRBOLES

La técnica DFS funciona muy bien sobre árboles, ya que nos permite primero recolectar información sobre los subárboles de mis hijos y luego agruparla acordemente



TEMPLATE

El código de DFS en un árbol siempre va a tener una estructura similar

```
int main(){

    int n;
    cin>>n; //leo el tamaño del arbol

    edges.resize(n); //ajusto tamaños
    visitado.resize(n, 0);
    d.resize(n);

    for(int i = 0; i<n-1; i++){
        int u, v;
        cin>>u>>v;
        u--; v--;
        edges[u].push_back(v);
        edges[v].push_back(u);
    }

    dfs(0); //hago dfs desde la raiz

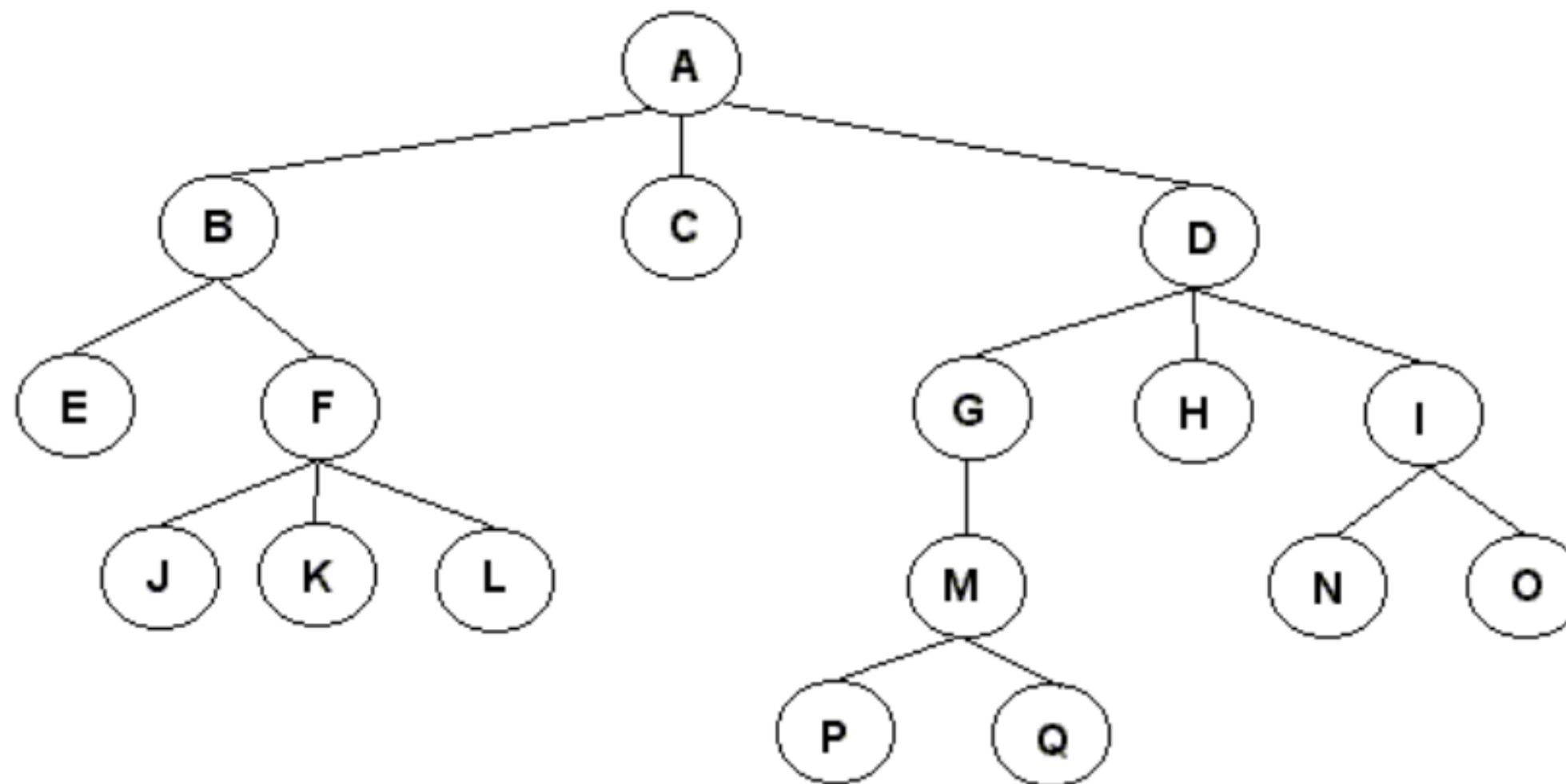
    //obtengo la respuesta con la data
    int maxi = 0;
    for(int i = 0; i<n; i++){
        maxi = max(maxi, d[i]);
    }
    cout<<maxi<<endl;
}
```

```
//creo donde guardare la informacion
vector<vector<int> > edges;
vector<int> visitado;
vector<int> d;

void dfs(int nodo){
    //hago cosas al entrar al nodo
    visitado[nodo] = 1;
    d[nodo] = 1;
    for(auto u: edges[nodo]){
        if(visitado[u] == 0){
            //recursiono a mis hijos
            dfs(u);
            //agrupo informacion
            d[nodo] = max(d[nodo], 1 + d[u]);
        }
    }
    //hago cosas a la salida
    visitado[nodo] = 2;
}
```

PROBLEMA

Dado un árbol con raíz, para cada nodo encuentra su padre, o -1 si no tiene padre.



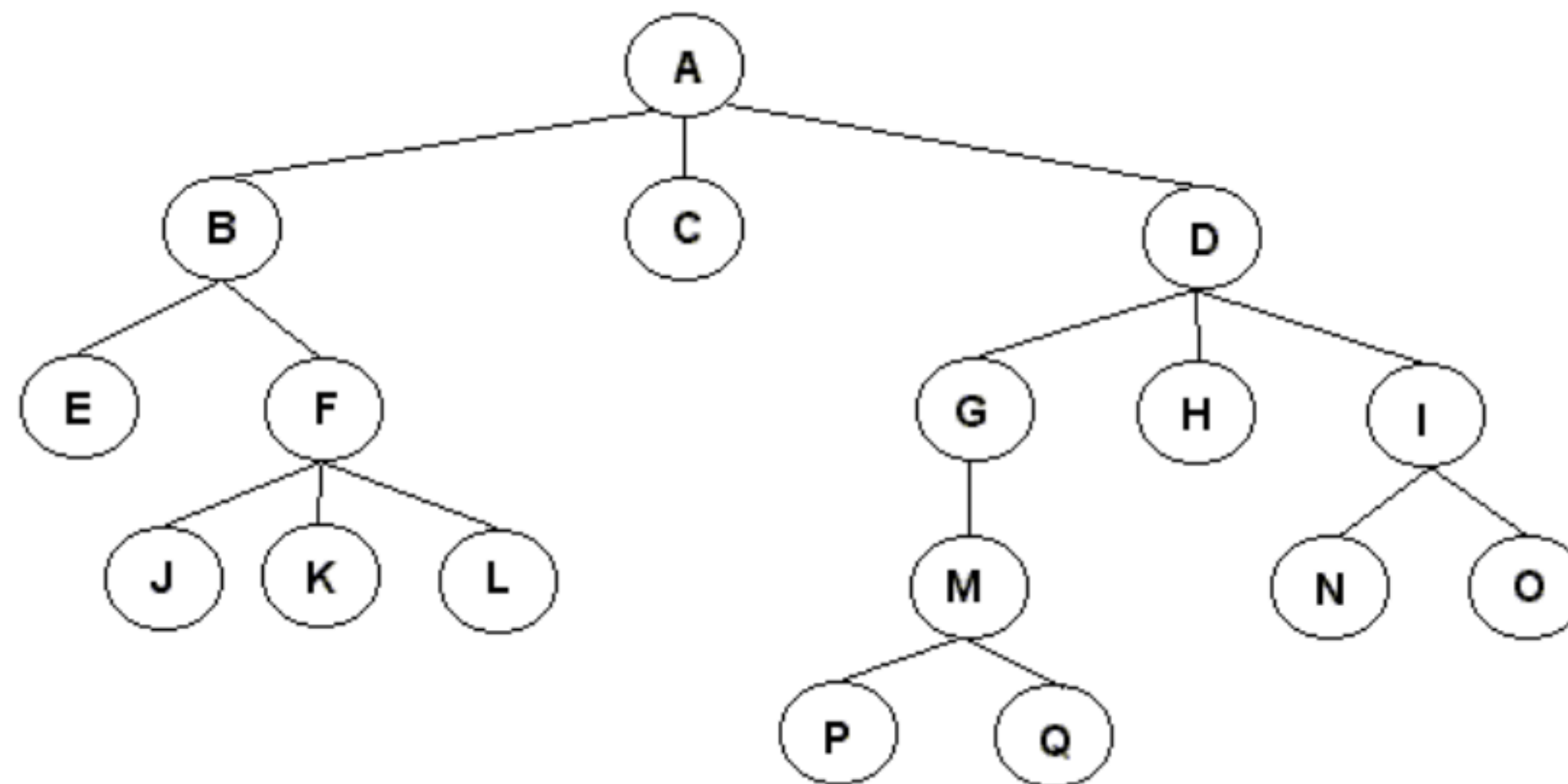
SOLUCIÓN

```
vector<vector<int> > edges;
vector<int> visitado;
vector<int> padre;

void dfs(int nodo){
    visitado[nodo] = 1;
    for(auto u: edges[nodo]){
        if(visitado[u] == 0){
            padre[u] = nodo;
            dfs(u);
        }
    }
    visitado[nodo] = 2;
}
```

PROBLEMA

Dado un árbol con raíz, para cada nodo encuentra el tamaño del subárbol correspondiente a dicho nodo



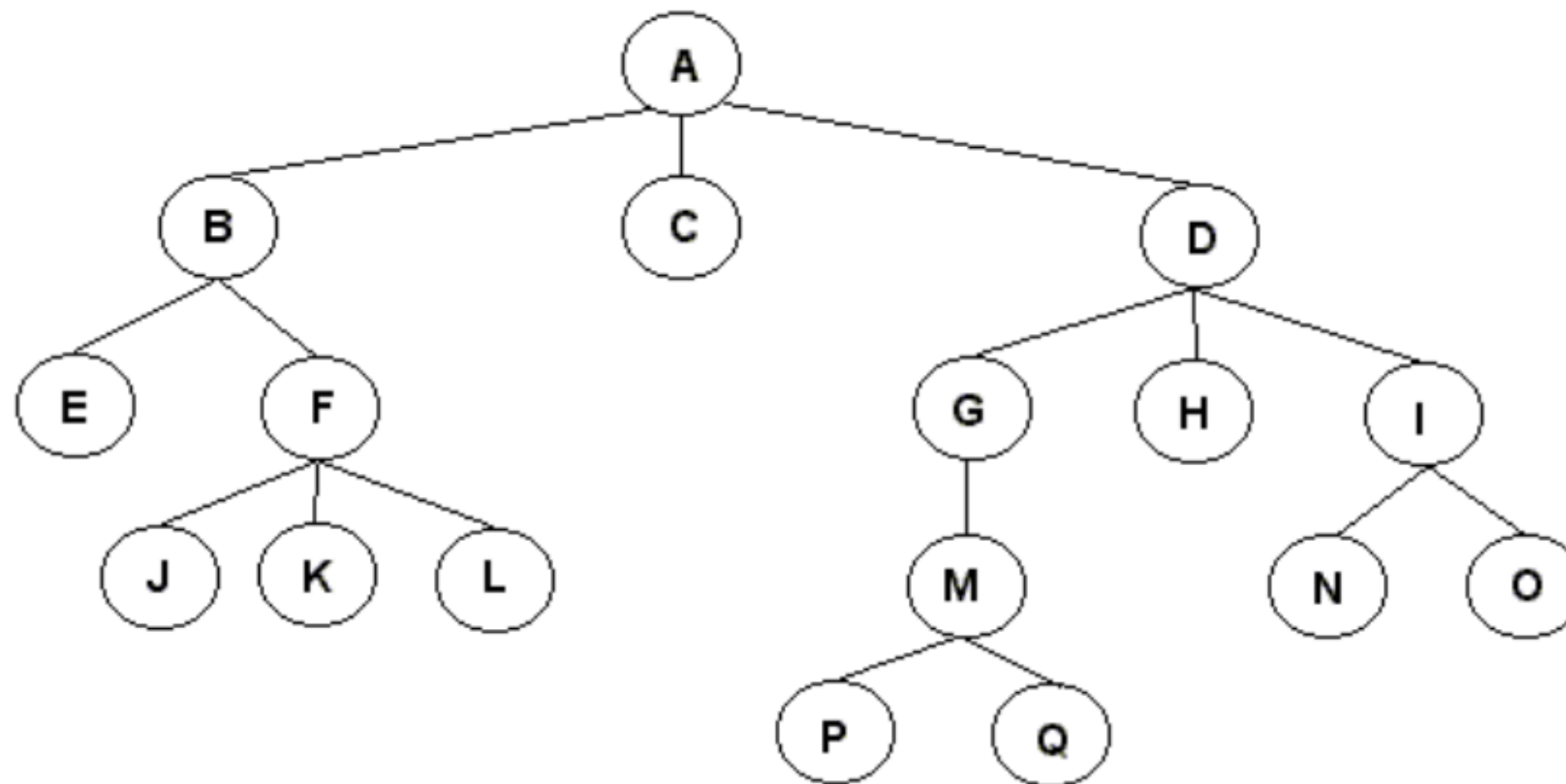
SOLUCIÓN

```
vector<vector<int> > edges;
vector<int> visitado;
vector<int> subtree;

void dfs(int nodo){
    visitado[nodo] = 1;
    subtree[nodo] = 1;
    for(auto u: edges[nodo]){
        if(visitado[u] == 0){
            dfs(u);
            subtree[nodo] += subtree[u];
        }
    }
    visitado[nodo] = 2;
}
```

PROBLEMA

Dado un árbol con raíz, para cada nodo encuentra el numero de hojas que tiene debajo e incluyéndome.



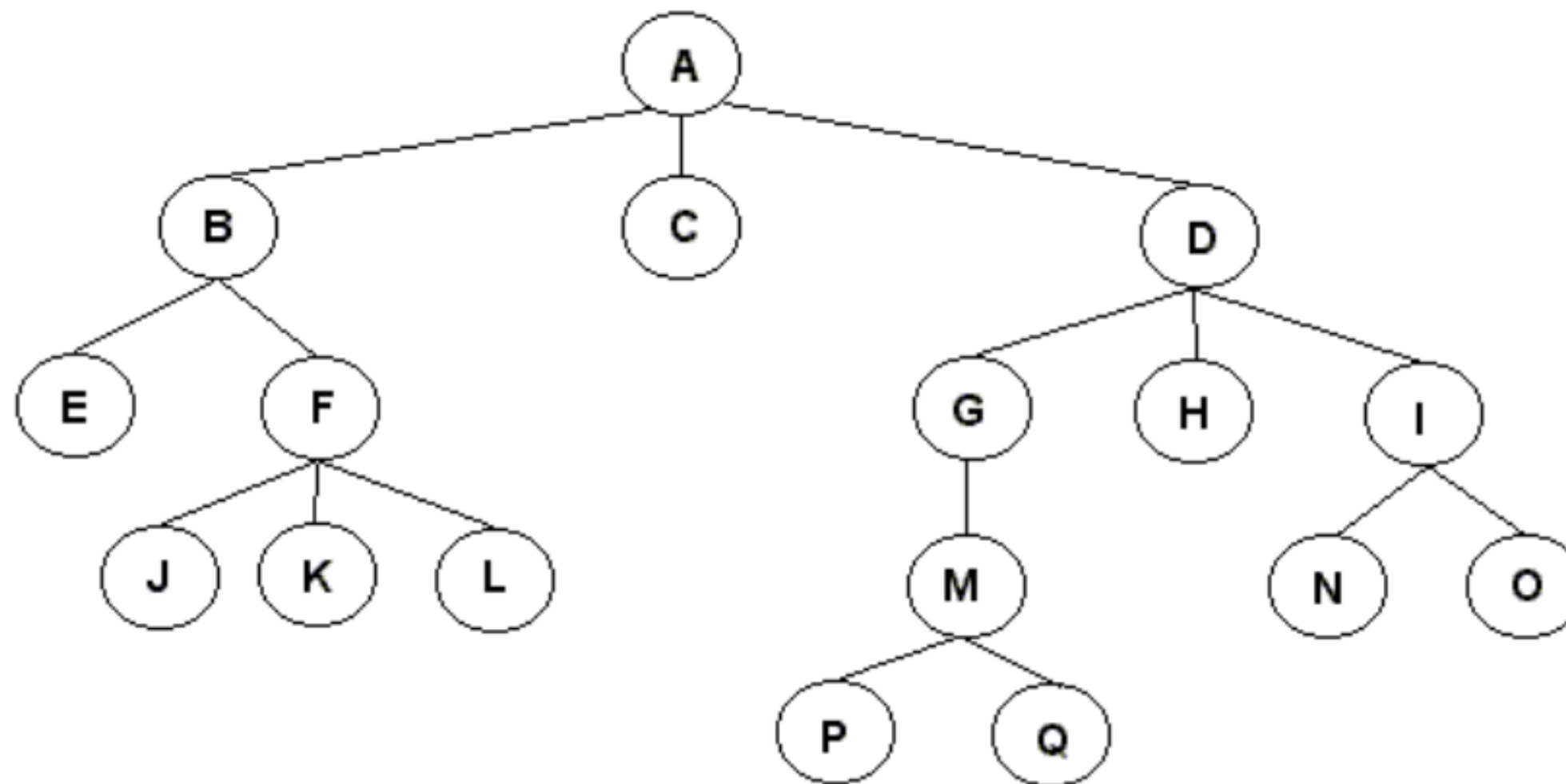
SOLUCIÓN

```
vector<vector<int> > edges;
vector<int> visitado;
vector<int> hojas;

void dfs(int nodo){
    visitado[nodo] = 1;
    hojas[nodo] = 0;
    for(auto u: edges[nodo]){
        if(visitado[u] == 0){
            dfs(u);
            hojas[nodo] += hojas[u];
        }
    }
    if(hojas[nodo] == 0){
        hojas[nodo] = 1;
    }
    visitado[nodo] = 2;
}
```

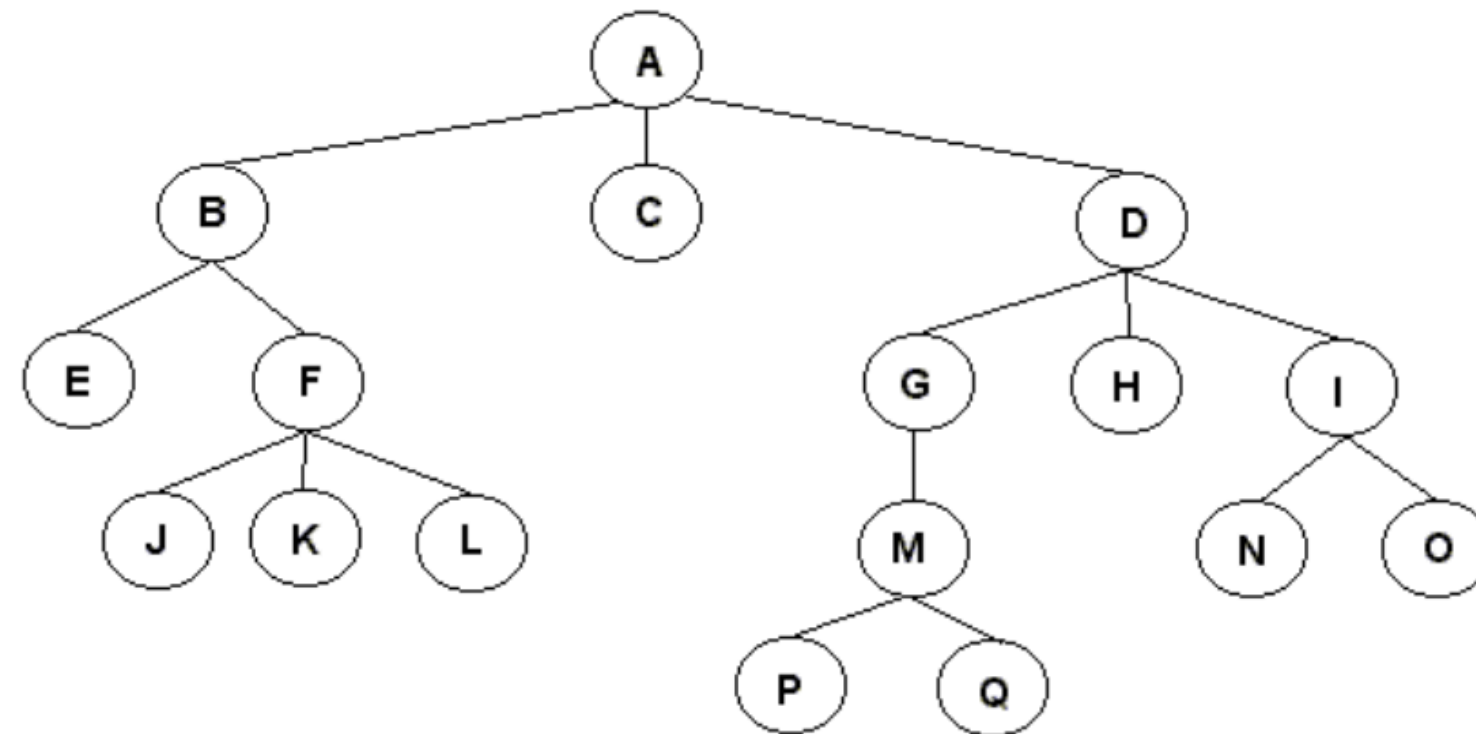
PROBLEMA

Dado un árbol con raíz, para cada nodo encuentra la cantidad máxima de nodos que puedo visitar moviendome hacia abajo.



APLANAR UN ARBOL CON DFS

Considera escribir (algún) orden en que el DFS recorre los nodos de un árbol



Aquí podría ser: **ABEFJKLCDGMPQHINO**

Notemos que cada subárbol queda de forma continua en este orden, lo cual nos podría ayudar para resolver algún tipo de problemas.